

P1233R1

Shift-by-negative in `shift_left` and `shift_right`

Published Proposal, 2018-11-25

Authors:

[Ashley Hedberg](#) (Google)

[Matt Calabrese](#) (Google)

[Bryce Adelstein Lelbach](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LWG

Table of Contents

- 1 Intro**
- 2 Revisions**
- 3 Concerns about current behavior**
- 4 Proposal**
- 5 Non-Proposals**
 - 5.1 Reverse shift when shifting by a negative n
 - 5.2 Changing behavior of shifting by large n
- 6 Wording**
- 7 Acknowledgements**

References

Informative References

§ 1. Intro

[\[P0769R2\]](#) was applied to the C++ working paper in Rapperswil. That paper defines shifting a range by a negative n as a no-op in item (7) of the design decisions section. The [LEWG discussion notes](#) from Albuquerque suggest that this design point was not discussed.

LEWG voted in San Diego to forward this proposal to LWG for C++20.

§ 2. Revisions

— R1 adds wording and removes the suggested polls section.

§ 3. Concerns about current behavior

The current treatment of a negative shift as a shift of 0 seems unlikely to match user intent and may hide bugs. If the programmer explicitly wrote a negative value, they probably didn't expect a shift of 0. If the user specified a negative shift as the result of some programmatic calculation, it is likely that the calculation was incorrect, or that a shift in the opposite direction would be the correct behavior. Either way, implicitly shifting by 0 feels questionable.

§ 4. Proposal

We propose that shifting a range by a negative `n` be a precondition violation; that is, `shift_left` and `shift_right` should require that `n` be greater than or equal to 0. This is consistent with `[expr.shift]`(<http://eel.is/c++draft/expr.shift>), which has a precondition that the right operand to `<<` and `>>` must be greater than or equal to 0. Compilers, static analyzers, and other analysis tools could more effectively warn programmers about such shifts if shifting by negative counts was a precondition violation.

§ 5. Non-Proposals

§ 5.1. Reverse shift when shifting by a negative `n`

Some users may expect a shift in the opposite direction when passing a negative `n` to `shift_left` and `shift_right`. The [LWG discussion notes](#) on P0769R2 suggest that there are APIs which do this; one example is [perlop](#). This could have a non-trivial cost and is inconsistent with `expr.shift`, so we do not propose it here.

§ 5.2. Changing behavior of shifting by large `n`

`expr.shift` has another precondition that the right operand must be less than the length in bits of the left operand. We do not propose changing `shift_left` and `shift_right` to have a similar precondition, as we believe it would be valuable to allow shifting all elements out of a range.

§ 6. Wording

Note: The following changes are relative to the post-Rapperswil 2018 working draft of ISO/IEC 14882, ([\[N4762\]](#)).

Note: The  character is used to denote a placeholder number which shall be selected by the editor.

Modify 23.6.14 [\[alg.shift\]](#) as follows:

23.6.14 Shift

[alg.shift]

```
template<class ForwardIterator>
constexpr ForwardIterator
    shift_left(ForwardIterator first, ForwardIterator last,
               typename iterator_traits<ForwardIterator>::difference_type
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    shift_left(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
               typename iterator_traits<ForwardIterator>::difference_type
```

- 1 **Requires/Mandates:** The type of **first* shall satisfy the *Cpp17MoveAssignable* requirements.

❖ **Expects:** $n \geq 0$.

- 2 **Effects:** If $n \leq 0$ or $n \geq \text{last} - \text{first}$, does nothing. Otherwise, moves the element from position $\text{first} + n + i$ into position $\text{first} + i$ for each non-negative integer $i < (\text{last} - \text{first}) - n$. In the first overload case, does so in order starting from $i = 0$ and proceeding to $i = (\text{last} - \text{first}) - n - 1$.
- 3 **Returns:** $\text{first} + (\text{last} - \text{first} - n)$ if n is positive and $n < \text{last} - \text{first}$, otherwise first if n is positive, otherwise last .
- 4 **Complexity:** At most $(\text{last} - \text{first}) - n$ assignments.

```
template<class ForwardIterator>
constexpr ForwardIterator
    shift_right(ForwardIterator first, ForwardIterator last,
               typename iterator_traits<ForwardIterator>::difference_type
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    shift_right(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
               typename iterator_traits<ForwardIterator>::difference_type
```

- 5 **Requires/Mandates:** The type of **first* shall satisfy the *Cpp17MoveAssignable* requirements. *ForwardIterator* shall meet the *Cpp17BidirectionalIterator* requirements ([\[bidirectional.iterators\]](#)) or the *Cpp17ValueSwappable* requirements.

❖ **Expects:** $n \geq 0$.

- 6 **Effects:** If $n \leq 0$ or $n \geq \text{last} - \text{first}$, does nothing. Otherwise, moves the element from position $\text{first} + i$ into position $\text{first} + n + i$ for each non-negative integer $i < (\text{last} - \text{first}) - n$.

$(\text{last} - \text{first}) - n$. In the first overload case, if *ForwardIterator* satisfies the *Cpp17BidirectionalIterator* requirements, does so in order starting from $i = (\text{last} - \text{first}) - n - 1$ and proceeding to $i = 0$.

7 Returns: $\text{first} + n$ if ~~n is positive and~~ $n < \text{last} - \text{first}$, otherwise last ~~if n is positive, otherwise first .~~

8 Complexity: At most $(\text{last} - \text{first}) - n$ assignments or swaps.

§ 7. Acknowledgements

- Bryce Adelstein Lelbach for the wording and Bikeshed formatting of this proposal.
- Dan Raviv and Casey Carter for feedback on an earlier draft of this proposal.

§ References

§ Informative References

[N4762]

Richard Smith. Working Draft, Standard for Programming Language C+. 7 July 2018. URL: <https://wg21.link/n4762>

[P0769R2]

Dan Raviv. Add shift to <algorithm>. 6 June 2018. URL: <https://wg21.link/p0769r2>